



**Universitatea Tehnică a Moldovei Facultatea Calculatoare, Informatică și
Microelectronică**

Departamentul Inginerie Software și Automatică

BOTNARI MARIA-ELENA, FAF-232

REPORT

***Laboratory Work nr. 1
at Formal Languages & Finite Automata***

Checked by:
Dumitru Crețu, *university assistant*
DISA, FCIM, UTM

Chișinău – 2025

Theory

A formal grammar is a set of rules used to generate strings in a specific language. It consists of non-terminal symbols (which can be replaced), terminal symbols (which appear in the final string), production rules (which define how symbols are replaced), and a start symbol.

A finite automaton (FA) is a computational model used to recognize patterns in strings. It consists of states, transitions, a start state, and accepting states. After converting a grammar into an FA, we can verify if a given string belongs to the language defined by the grammar.

Objectives

- Implement a class to represent a grammar.
- Generate five valid strings based on the grammar rules.
- Convert the grammar into a finite automaton.
- Check if a given string can be derived using the finite automaton.

Implementation Description

Grammar Class: The Grammar class is designed to represent a formal grammar used to generate strings based on a set of rules. When the class is initialized, it takes four parameters: V_n (a list of non-terminal symbols, which are variables or placeholders), V_t (a list of terminal symbols, which are the actual symbols that appear in the generated strings), P (the production rules that define how non-terminals can be replaced with terminals or other non-terminals), and S (the start symbol, which is the non-terminal from which the generation begins). The class stores these parameters, and then it processes the production rules (P) by splitting the rules into pairs. Each pair contains a non-terminal symbol (like S) and its corresponding replacement rules (like $aS \mid bD$). The rules are stored in a dictionary called $P_dictionary$, where each key is a non-terminal symbol, and its value is a list of possible replacements. If a rule involves alternatives (like $aS \mid bD$), it splits them into separate entries in the dictionary. This allows the grammar to easily generate strings by selecting random production rules for each non-terminal symbol encountered.

```

pairs = self.P.split(", ")
for pair in pairs:
    a, b = pair.split("->")
    if "|" in b:
        rules = b.split("|")
    else:
        rules = [b]

    if a in self.P_dictionary:
        self.P_dictionary[a].extend(rules)
    else:
        self.P_dictionary[a] = rules

```

Fig. 1: Snippet of code from the Grammar Class

Generating Strings: The `generate_string` function is responsible for generating a string from the grammar. It starts by taking the start symbol (S) and repeatedly applies production rules to replace non-terminal symbols with terminal symbols or other non-terminals. The process begins with the string set to the start symbol (S). Inside the function, a loop checks if the current string still contains any non-terminal symbols (from V_n). If there are non-terminals, the loop iterates through the string character by character. For each character, if it is a non-terminal, the function looks up the corresponding production rules in the `P_dictionary` and randomly selects one to replace the non-terminal. This replacement results in a new string that may still contain non-terminals, so the loop continues, repeating the replacement process until no more non-terminals are left. When the string contains only terminal symbols, the loop ends, and the function returns the final string along with the list of transformation steps it took to generate the string. These steps show the progression from the original string to the final string, providing insight into how the grammar's production rules were applied at each stage.

```

while any(char in self.Vn for char in string):
    new_string = ""
    for char in string:
        if char in self.P_dictionary:
            new_string += random.choice(self.P_dictionary[char])
        else:
            new_string += char
    steps.append(f"{string} -> {new_string}")
    string = new_string

```

Fig. 2: Snippet of code from the `generate_string` function

Converting Grammar to FA: The `convert_grammar_to_fa` function transforms the given grammar into a finite automaton (FA), a computational model used to recognize strings. This function starts by creating a set of states, which corresponds to the non-terminal symbols in the grammar (V_n). It also creates an empty dictionary for the transitions and a set for the accepting states. For each production rule in the grammar, the function checks if the right-hand side of the rule ends with a terminal symbol. If it does, the left-hand side non-terminal symbol is added to the accepting states, since the rule produces a terminal string and can be considered an accepted final state. If the right-hand side of the rule consists of a terminal symbol followed by a non-terminal, a transition is recorded in the transitions dictionary. The transition shows that from the current non-terminal (the left-hand side), reading the terminal symbol moves the automaton to another state, represented by the non-terminal on the right-hand side. This process repeats for all production rules, building up the states, transitions, and accepting states of the finite automaton. Finally, the function returns the set of states, the transitions, the accepting states, and the start state (S), providing a complete finite automaton based on the grammar.

Checking String in FA: The `check_string_via_transition` function simulates how a finite automaton processes a string. It starts by converting the grammar into a finite automaton using the `convert_grammar_to_fa` function. This gives the set of states, the transitions, and the accepting states. The string to be checked is processed character by character, starting from the start state. For each character in the string, the function checks if there is a valid transition from the current state to the next state, based on the current symbol. If a transition exists, the automaton moves to the next state, and this step is recorded. If no valid transition exists for the current symbol, the string is immediately rejected, and the function returns the steps showing that the string was rejected. If the string is processed successfully and ends in an accepting state, the string is considered accepted by the finite automaton. The function then returns a detailed list of steps, showing each state transition as the automaton processes the string. If the string ends in a non-accepting state, the string is rejected, and the steps indicate where the process failed.

```

for symbol in string:
    if (current_state, symbol) in transitions:
        next_state = transitions[(current_state, symbol)]
        steps.append(f"Read '{symbol}', move from {current_state} to {next_state}")
        current_state = next_state
    else:
        steps.append(f"Read '{symbol}', no transition exists. String rejected.")
        return steps

if current_state in accepting_states:
    steps.append(f"String accepted: Ended in accepting state {current_state}.")
else:
    steps.append(f"String rejected: Ended in non-accepting state {current_state}.")

```

Fig. 3: Snippet of code from the check_string_via_transition function

Results:

```

The steps taken to generate bcd: ['S -> bD', 'bD -> bcD', 'bcD -> bcd']
The steps taken to generate abdf: ['S -> aS', 'aS -> abD', 'abD -> abdR', 'abdR -> abdf']
The steps taken to generate fbf: ['S -> fR', 'fR -> fbR', 'fbR -> fbf']
The steps taken to generate bcdf: ['S -> bD', 'bD -> bcD', 'bcD -> bcdR', 'bcdR -> bcdf']
The steps taken to generate ff: ['S -> fR', 'fR -> ff']

5 Unique Strings: {'fbf', 'bcd', 'abdf', 'ff'}

```

Fig. 4: The strings generated

```

Converting the object of type Grammar to type Finite Automaton
States: {'S', 'R', 'D'}
Transitions: {('S', 'a'): 'S', ('S', 'b'): 'D', ('S', 'f'): 'R', ('D', 'c'): 'D', ('D', 'd'): 'R', ('R', 'b'): 'R'}
Terminals: {'R', 'D'}
Start state: S

```

Fig. 5: The Finite Automaton object

```

Checking if the string aaaabd can be obtained via the state transition from FA:
['Start at S', "Read 'a', move from S to S", "Read 'a', move from S to S", "Read 'a', move from S to S", "Read 'a', move from S to S", "Read 'b', move from S to D", "Read 'd', move from D to R", 'String accepted: Ended in accepting state R.']

```

Fig. 6: The results after checking a string via transitions

Conclusion

In conclusion, this code successfully converts a formal grammar into a finite automaton and checks if a string belongs to the language. The Grammar class stores the grammar rules, generates valid strings, and tracks the transformation steps. The FA class builds a finite automaton using the grammar's states and transitions, allowing for structured string validation. By simulating state transitions, we can determine whether a given string follows the defined rules. This approach helped me understand how grammars and finite automata work together, making it easier to process and verify structured languages.